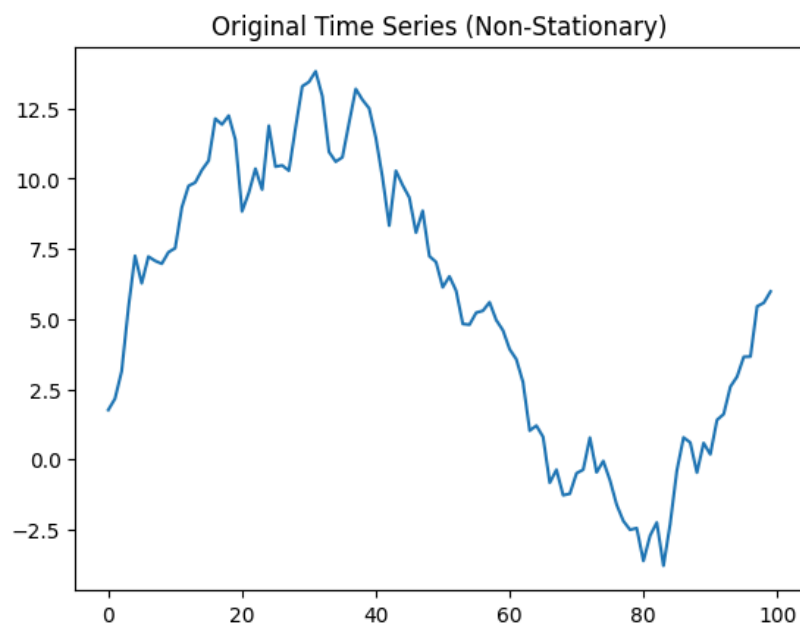


```
# Time Series Analysis Practical
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
```

```
# Create Sample Time Series Data
np.random.seed(0)
data = np.cumsum(np.random.randn(100))
ts = pd.Series(data)

plt.figure()
plt.plot(ts)
plt.title("Original Time Series (Non-Stationary)")
plt.show()
```

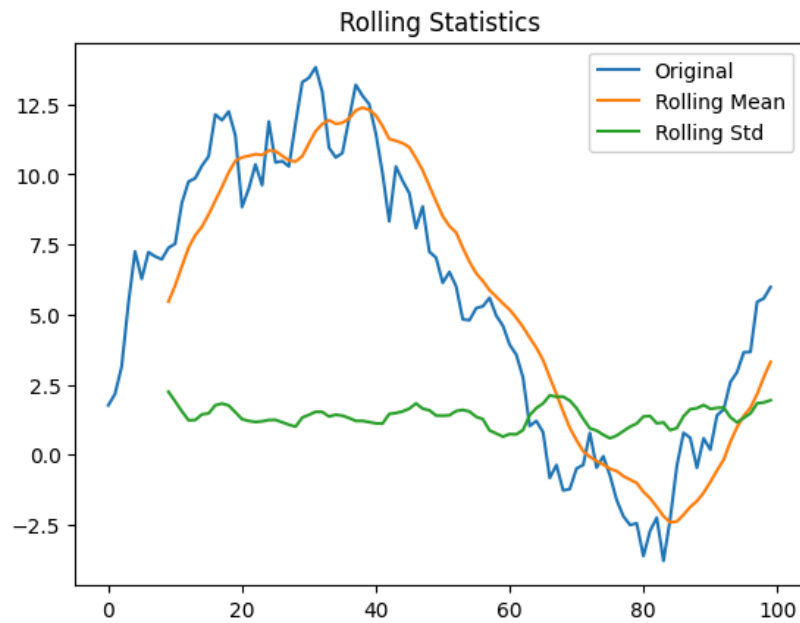


```
# (a) CHECK STATIONARITY
print("\n--- Checking Stationarity ---")

rolling_mean = ts.rolling(window=10).mean()
rolling_std = ts.rolling(window=10).std()

plt.figure()
plt.plot(ts, label='Original')
plt.plot(rolling_mean, label='Rolling Mean')
plt.plot(rolling_std, label='Rolling Std')
plt.legend()
plt.title("Rolling Statistics")
plt.show()
```

--- Checking Stationarity ---



```
# (c) DICKEY-FULLER TEST
print("\n--- Dickey-Fuller Test ---")

result = adfuller(ts)

print("ADF Statistic:", result[0])
print("p-value:", result[1])

if result[1] < 0.05:
    print("Data is Stationary")
else:
    print("Data is NOT Stationary")
```

```
--- Dickey-Fuller Test ---
ADF Statistic: -1.1320384625097901
p-value: 0.7021277385898382
Data is NOT Stationary
```

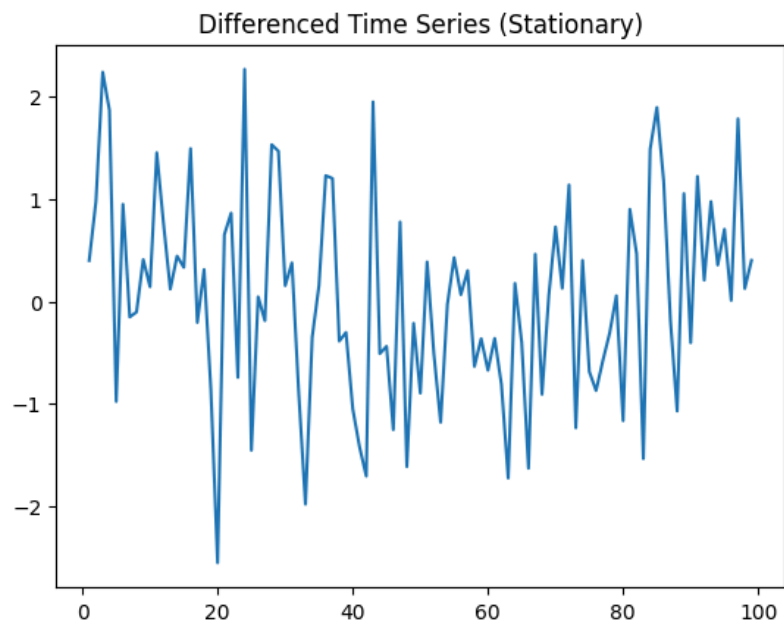
```
# (b) MAKE DATA STATIONARY
print("\n--- Converting to Stationary ---")

ts_diff = ts.diff().dropna()

plt.figure()
plt.plot(ts_diff)
plt.title("Differenced Time Series (Stationary)")
plt.show()

result_diff = adfuller(ts_diff)
print("New p-value:", result_diff[1])
```

--- Converting to Stationary ---



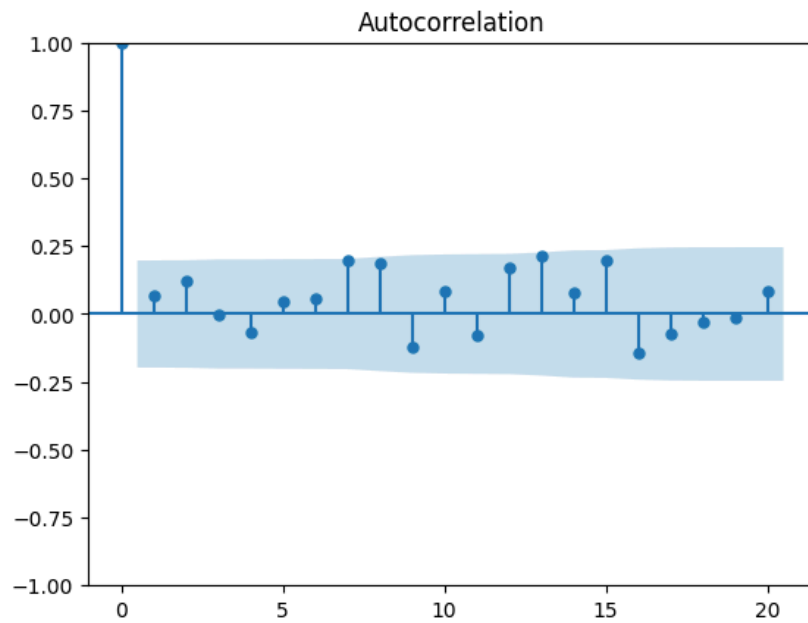
```
# (d) ACF and PACF PLOTS
print("\n--- ACF and PACF ---")

plt.figure()
plot_acf(ts_diff)
plt.show()

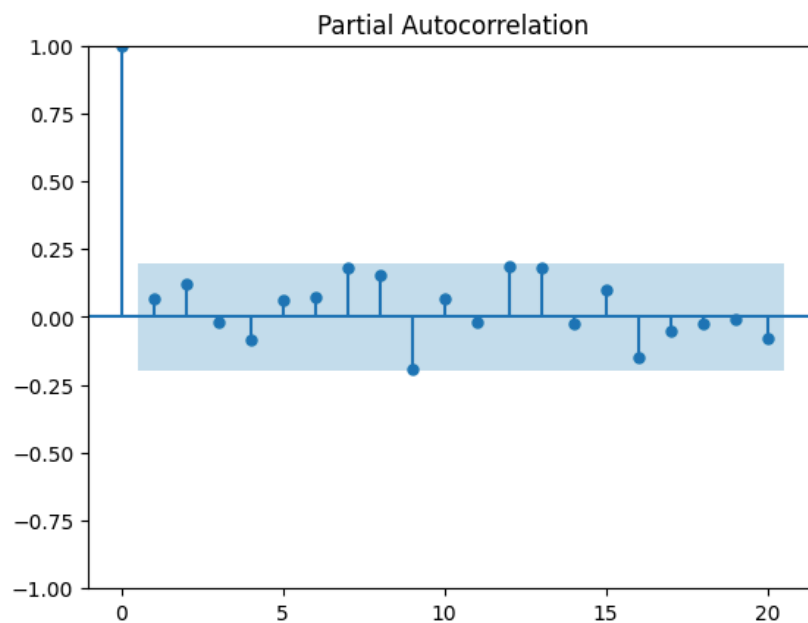
plt.figure()
plot_pacf(ts_diff)
plt.show()
```

--- ACF and PACF ---

<Figure size 640x480 with 0 Axes>



<Figure size 640x480 with 0 Axes>



```
# (e) ARIMA MODEL
print("\n--- ARIMA Model ---")

model = ARIMA(ts, order=(1,1,1))
model_fit = model.fit()

print(model_fit.summary())

plt.figure()
plt.plot(ts, label='Original')
plt.plot(model_fit.fittedvalues, color='red', label='Fitted')
plt.legend()
plt.title("ARIMA Model Fit")
plt.show()
```

--- ARIMA Model ---

SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:      100
Model:                 ARIMA(1, 1, 1)      Log Likelihood      -139.082
Date:                 Fri, 03 Apr 2026      AIC      284.163
Time:                 08:08:46      BIC      291.949
Sample:              0      HQIC      287.313
                    - 100
Covariance Type:      opg
=====
```

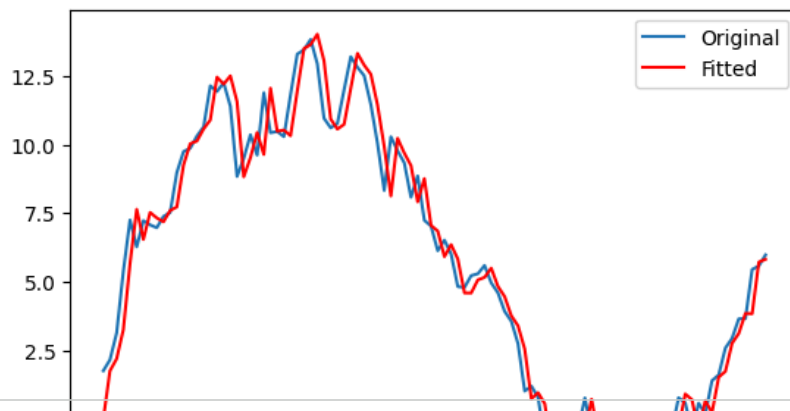
	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.9450	0.082	11.491	0.000	0.784	1.106
ma.L1	-0.8819	0.115	-7.687	0.000	-1.107	-0.657
sigma2	0.9707	0.147	6.591	0.000	0.682	1.259

```
=====
Ljung-Box (L1) (Q):      0.04      Jarque-Bera (JB):      0.19
Prob(Q):                 0.84      Prob(JB):      0.91
Heteroskedasticity (H):  0.64      Skew:      -0.02
Prob(H) (two-sided):     0.21      Kurtosis:      2.79
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

ARIMA Model Fit



```
# (f) FORECASTING
print("\n--- Forecasting ---")

forecast = model_fit.forecast(steps=10)

plt.figure()
plt.plot(ts, label='Original')
```